**UNIVERSIDADE FUMEC**
**CURSO DE CIÊNCIA DA COMPUTAÇÃO**
**DESENVOLVIMENTO WEB FRONT-END**

**DAVI TEIXEIRA**

# SKYFARE:
## planejador de viagens com previsão do tempo e câmbio

**BELO HORIZONTE**
**2026**

**DAVI TEIXEIRA**

# SKYFARE:
# planejador de viagens com previsão do tempo e câmbio

Relatório parcial apresentado à disciplina Desenvolvimento Web Front-End do curso de Ciência da Computação da Universidade FUMEC, como requisito para avaliação do Trabalho 00 – Proposta.

Professor: Dângelo Silva Miranda

**BELO HORIZONTE**

**2026**

**SUMÁRIO**

**1 INTRODUÇÃO.....3**
**2 IDENTIFICAÇÃO DO PROJETO.....4**
2.1 Nome da aplicação.....4
2.2 Tema.....4
2.3 Integrantes do grupo.....4
2.4 Repositório do projeto.....4
**3 DESCRIÇÃO DA APLICAÇÃO.....5**
3.1 Funcionamento geral.....5
3.2 APIs consumidas.....5
3.3 Dados armazenados.....6
3.4 Tecnologias.....6
**4 TELAS DA APLICAÇÃO.....7**
**5 REQUISITOS.....8**
5.1 Requisitos funcionais.....8
5.2 Requisitos não funcionais.....9
5.3 Dados do dashboard de relatórios.....10
**6 PLANEJAMENTO.....11**
**REFERÊNCIAS.....12**

# 1 INTRODUÇÃO

Este relatório apresenta a proposta do projeto que será desenvolvido ao longo do semestre na disciplina Desenvolvimento Web Front-End, do curso de Ciência da Computação da Universidade FUMEC. O documento corresponde ao Trabalho 00, etapa de planejamento, e reúne o nome e o tema da aplicação, o integrante responsável, a quantidade de telas, os requisitos funcionais e o endereço do repositório do projeto.

A aplicação proposta, chamada Skyfare, é um planejador de viagens. A partir do nome de uma cidade, ela mostra em uma única tela a previsão do tempo no destino e a cotação da moeda local em relação ao real. O usuário também pode cadastrar as viagens que pretende fazer, acompanhar o orçamento convertido para a moeda do país e consultar relatórios sobre os seus planos.

As tarefas do projeto foram registradas em um quadro Kanban, entregue em planilha junto com este relatório. As seções seguintes descrevem a aplicação, as suas telas, os seus requisitos e o cronograma previsto.

# 2 IDENTIFICAÇÃO DO PROJETO

## 2.1 Nome da aplicação

O nome escolhido é Skyfare. A palavra junta os termos em inglês *sky* (céu), que remete à previsão do tempo, e *fare* (tarifa), que remete ao custo da viagem. Assim, o nome resume as duas informações centrais da aplicação: o clima do destino e o valor da moeda local.

## 2.2 Tema

O tema é o planejamento de viagens, nacionais e internacionais. Quem planeja uma viagem costuma consultar vários sites para saber como estará o tempo no destino e quanto vale a moeda local. O Skyfare reúne essas consultas em um só lugar e permite guardar as viagens planejadas, com datas e orçamento, para acompanhá-las até o embarque.

## 2.3 Integrantes do grupo

O projeto é desenvolvido individualmente pelo aluno Davi Teixeira, responsável por todas as etapas: planejamento, prototipação, implementação, testes e apresentação.

## 2.4 Repositório do projeto

O código-fonte está disponível no GitHub, no seguinte endereço:

**https://github.com/DaviTeixeiraR/skyfare**

O repositório segue o fluxo GitFlow, com as *branches* main e develop e uma *branch* feature/* para cada funcionalidade. Os *commits* são atômicos e as mensagens seguem o padrão Conventional Commits.

# 3 DESCRIÇÃO DA APLICAÇÃO

## 3.1 Funcionamento geral

O usuário digita o nome de uma cidade e recebe sugestões enquanto escreve. Ao escolher uma delas, a aplicação obtém as coordenadas e o país da cidade e, em seguida, faz duas consultas em paralelo: uma busca o clima atual e a previsão para os próximos sete dias; a outra identifica a moeda do país e busca a sua cotação em reais.

O resultado aparece em dois cartões independentes, um para o clima e outro para o câmbio. Cada cartão tem o seu próprio estado de carregamento e de erro. Assim, se o serviço de cotação ficar indisponível, o clima continua sendo exibido normalmente, e vice-versa.

Usuários autenticados também podem cadastrar viagens, informando o destino, as datas de ida e volta e o orçamento em reais. Na tela de detalhe de cada viagem, o orçamento é convertido automaticamente para a moeda do destino com a cotação do dia.

## 3.2 APIs consumidas

A aplicação consome quatro APIs públicas e gratuitas, que não exigem chave de acesso, e uma API local criada com o JSON-Server. O Quadro 1 resume o papel de cada uma.

Quadro 1 – APIs consumidas pela aplicação

| API | Finalidade | Endereço |
|---|---|---|
| Open-Meteo Geocoding | Sugerir cidades e obter as coordenadas e o código do país | geocoding-api.open-meteo.com |
| Open-Meteo Forecast | Obter o clima atual e a previsão de sete dias | api.open-meteo.com |
| REST Countries | Identificar a moeda, a bandeira e o continente do país | restcountries.com |
| AwesomeAPI de Moedas | Obter a cotação atual e o histórico de 30 dias em reais | economia.awesomeapi.com.br |
| JSON-Server | Armazenar usuários, viagens e favoritos | localhost (API local) |

Fonte: Elaborado pelo autor (2026).

A API de geocodificação não informa a moeda do país. Por isso, o código do país passa pela REST Countries antes da consulta de cotação. Essa cadeia de chamadas dependentes é organizada com operadores da biblioteca RxJS, como *debounceTime*, *switchMap*, *forkJoin* e *catchError*.

### 3.3 Dados armazenados

O JSON-Server guarda três coleções: *users* (nome, e-mail, senha, perfil e situação da conta), *trips* (usuário, cidade, país, moeda, coordenadas, datas, orçamento e observações) e *favorites* (destinos favoritos de cada usuário). O histórico das últimas buscas fica no armazenamento local do navegador (*localStorage*).

### 3.4 Tecnologias

A aplicação é construída com o *framework* Angular e a linguagem TypeScript. São usados componentes *standalone* com comunicação por entradas e saídas, serviços com injeção de dependência e *HttpClient*, *signals* para o estado da aplicação, formulários reativos, *guards* de rota, *interceptors* HTTP e *pipes* customizados. Os gráficos do dashboard são feitos com a biblioteca Chart.js, e o controle de versão usa Git e GitHub.

## 4 TELAS DA APLICAÇÃO

A aplicação terá 10 (dez) telas, descritas no Quadro 2. A coluna Acesso indica quem pode abrir cada tela: as telas públicas não exigem login, as telas de usuário exigem login e a tela de administração exige o perfil de administrador.

Quadro 2 – Telas da aplicação

| Nº | Tela | Descrição | Acesso |
|---|---|---|---|
| 1 | Login | Entrada no sistema com e-mail e senha | Pública |
| 2 | Cadastro de usuário | Formulário de criação de conta | Pública |
| 3 | Busca rápida (início) | Busca de cidade com sugestões e cartões de clima e de câmbio | Pública |
| 4 | Minhas viagens | Lista das viagens do usuário, com filtros por situação e ordenação | Usuário |
| 5 | Detalhe da viagem | Clima, previsão de sete dias, cotação, conversor de valores e orçamento convertido | Usuário |
| 6 | Formulário de viagem | Cadastro e edição de uma viagem | Usuário |
| 7 | Dashboard | Relatórios com indicadores e gráficos | Usuário |
| 8 | Gerenciamento de usuários | Lista de usuários, alteração de perfil e ativação de contas | Administrador |
| 9 | Acesso negado | Aviso exibido quando o usuário não tem permissão | Pública |
| 10 | Página não encontrada | Aviso para endereços inexistentes (erro 404) | Pública |

Fonte: Elaborado pelo autor (2026).

As telas 4 a 8 são carregadas sob demanda (*lazy loading*): o código de cada área só é baixado pelo navegador quando o usuário a acessa pela primeira vez. As rotas dessas telas são protegidas por *guards*, que levam o visitante não autenticado ao login e o usuário sem permissão à tela de acesso negado.

# 5 REQUISITOS

## 5.1 Requisitos funcionais

Requisitos funcionais descrevem o que o sistema deve fazer. O Quadro 3 lista os requisitos da aplicação e a prioridade de cada um. Os requisitos de prioridade alta formam a versão mínima do sistema, e os demais serão implementados em seguida.

Quadro 3 – Requisitos funcionais

| Código | Requisito | Descrição | Prioridade |
|---|---|---|---|
| RF01 | Cadastrar usuário | O visitante cria uma conta informando nome, e-mail e senha. Os campos são validados e não é permitido repetir um e-mail já cadastrado. | Alta |
| RF02 | Autenticar usuário | O usuário entra com e-mail e senha e sai do sistema quando quiser. A sessão é mantida ao recarregar a página. | Alta |
| RF03 | Controlar acesso | Rotas restritas exigem login e a área de administração exige o perfil de administrador. Quem não tem permissão é levado ao login ou à tela de acesso negado. | Alta |
| RF04 | Buscar cidade | O sistema sugere cidades enquanto o usuário digita, mostrando estado e país para diferenciar cidades com o mesmo nome. | Alta |
| RF05 | Exibir clima atual | O sistema mostra a temperatura, a condição do tempo, a umidade e o vento no destino. | Alta |
| RF06 | Exibir cotação | O sistema mostra quanto vale uma unidade da moeda local em reais. Para destinos no Brasil, informa que não há conversão. | Alta |
| RF07 | Tratar falhas parciais | Os cartões de clima e de câmbio têm carregamento e mensagem de erro próprios, e a falha de um não afeta o outro. | Alta |
| RF08 | Cadastrar viagem | O usuário cadastra uma viagem com destino, data de ida, data de volta, orçamento em reais e observações. A data de volta não pode ser anterior à de ida. | Alta |
| RF09 | Listar viagens | O usuário vê as suas viagens, com filtros (futuras e realizadas) e ordenação por data ou por orçamento. | Alta |
| RF10 | Detalhar viagem | O sistema exibe os dados da viagem com o clima do destino, a cotação do dia e o orçamento convertido. | Alta |
| RF11 | Gerar relatórios | O dashboard apresenta indicadores e gráficos sobre as viagens e as moedas dos destinos (ver seção 5.3). | Alta |

| Código | Requisito | Descrição | Prioridade |
|---|---|---|---|
| RF12 | Editar e excluir viagem | O usuário altera os dados de uma viagem ou a exclui após confirmar a ação. | Média |
| RF13 | Exibir previsão de 7 dias | O sistema mostra as temperaturas mínima e máxima e a condição prevista para os próximos sete dias. | Média |
| RF14 | Converter valores | O usuário informa um valor em reais e vê o equivalente na moeda local, e vice-versa. | Média |
| RF15 | Exibir histórico da cotação | O sistema mostra, em gráfico, a variação da cotação da moeda nos últimos 30 dias. | Média |
| RF16 | Gerenciar usuários | O administrador lista os usuários, altera o perfil e ativa ou desativa contas. | Média |
| RF17 | Favoritar destinos | O usuário marca cidades como favoritas para acessá-las rapidamente. | Baixa |
| RF18 | Guardar histórico de buscas | O sistema guarda as últimas cidades pesquisadas para que possam ser consultadas de novo. | Baixa |

Fonte: Elaborado pelo autor (2026).

## 5.2 Requisitos não funcionais

Requisitos não funcionais descrevem qualidades e restrições do sistema. O Quadro 4 apresenta os definidos para o Skyfare.

Quadro 4 – Requisitos não funcionais

| Código | Categoria | Descrição |
|---|---|---|
| RNF01 | Desempenho | Rotas carregadas sob demanda (lazy loading), gráficos com carregamento adiado (@defer) e detecção de mudanças OnPush. |
| RNF02 | Cache | Cotações e previsões guardadas por alguns minutos, para evitar requisições repetidas às APIs. |
| RNF03 | Responsividade | A interface deve funcionar em celulares, tablets e computadores. |
| RNF04 | Usabilidade | Indicação de carregamento durante as consultas e mensagens de erro claras para o usuário. |
| RNF05 | Tipagem | Interfaces TypeScript para as respostas de todas as APIs consumidas. |
| RNF06 | Versionamento | Git e GitHub, com fluxo GitFlow, commits atômicos e mensagens no padrão Conventional Commits. |
| RNF07 | Segurança | A autenticação é simulada com o JSON-Server, adequada ao escopo acadêmico. Em produção, seria necessário um back-end com senhas criptografadas e tokens assinados. |

Fonte: Elaborado pelo autor (2026).

### 5.3 Dados do dashboard de relatórios

O dashboard reúne as seguintes informações sobre as viagens do usuário:

a) quantidade de viagens planejadas, em andamento e realizadas;
b) orçamento total planejado, em reais, e a média por viagem;
c) número de viagens por mês, em gráfico de barras;
d) distribuição das viagens por continente e por país, em gráfico de rosca;
e) variação da cotação, nos últimos 30 dias, das moedas dos próximos destinos, em gráfico de linhas;
f) próximas viagens, com contagem regressiva de dias e temperatura prevista;
g) cidades mais pesquisadas pelo usuário.

Para o administrador, o dashboard também mostra o número de usuários cadastrados e ativos e o total de viagens registradas no sistema.

## 6 PLANEJAMENTO

As tarefas foram organizadas em um quadro Kanban, entregue em planilha junto com este relatório. O quadro tem 29 tarefas, cada uma com descrição, itens de verificação, prioridade e data prevista de conclusão. O Quadro 5 resume as etapas. As datas são estimativas e serão ajustadas aos prazos definidos pelo professor no Microsoft Teams.

Quadro 5 – Cronograma previsto

| Etapa | Entregas | Conclusão prevista |
|---|---|---|
| Trabalho 00 – Planejamento | Kanban, relatório parcial e repositório no GitHub | 17/10/2026 |
| Trabalho 01 – Protótipo | Protótipo das 10 telas no Figma | 31/10/2026 |
| Estrutura do projeto | Projeto Angular, JSON-Server, interfaces, serviços de API e autenticação | 14/11/2026 |
| Desenvolvimento das telas | Telas 1 a 10, favoritos e histórico de buscas | 28/11/2026 |
| Otimização e testes | Lazy loading, cache, responsividade e casos de borda | 02/12/2026 |
| Entrega final | Relatório final e apresentação | 07/12/2026 |

Fonte: Elaborado pelo autor (2026).

# REFERÊNCIAS

ANGULAR. **Angular documentation**. [S. l.], 2026. Disponível em: https://angular.dev. Acesso em: 17 out. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

ATLASSIAN. **Comparing Git workflows**. [S. l.], 2026. Disponível em: https://www.atlassian.com/git/tutorials/comparing-workflows. Acesso em: 17 out. 2026.

AWESOMEAPI. **API de cotações de moedas**. [S. l.], 2026. Disponível em: https://docs.awesomeapi.com.br/api-de-moedas. Acesso em: 17 out. 2026.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. [S. l.], 2026. Disponível em: https://www.conventionalcommits.org/pt-br/v1.0.0/. Acesso em: 17 out. 2026.

OPEN-METEO. **Geocoding API**. [S. l.], 2026. Disponível em: https://open-meteo.com/en/docs/geocoding-api. Acesso em: 17 out. 2026.

OPEN-METEO. **Weather Forecast API**. [S. l.], 2026. Disponível em: https://open-meteo.com/en/docs. Acesso em: 17 out. 2026.

REST COUNTRIES. **REST Countries API**. [S. l.], 2026. Disponível em: https://restcountries.com. Acesso em: 17 out. 2026.

TYPICODE. **JSON Server**. [S. l.], 2026. Disponível em: https://github.com/typicode/json-server. Acesso em: 17 out. 2026.